00 / 21 — Phase 0 Assessment: Web and JavaScript Foundations

This assessment covers lessons 01 through 20. It decides whether you are ready for Phase 1 — React fundamentals. It is deliberately harder than a recall quiz: most items ask you to **predict, diagnose, or decide**, because Phases 1–3 assume you can reason about closures, identity, the event loop, and memory without looking things up.

Recommended time	3.5–4.5 hours in one or two sittings
Format	Closed-book first pass (no editor, no docs); then optionally run code to check yourself *before* reading the key
Total points	150
Pass	≥ 80% overall (120/150) **and** ≥ 70% in each of the four categories (see §7)

How to use it. Write answers on paper or in a plain text file. For code questions write the code by hand, then run it. Do not read §6 (answer key) until you have attempted every part. Score yourself with §7, honestly: partial credit rules are explicit.

Conventions. Unless stated otherwise: code runs in a modern browser as an ES module (strict mode), TypeScript is 5.x with `"strict": true`, and "the main thread" means the document's single main thread (Lesson 01).

Contents

1. Part A — Conceptual questions (40 pts)
2. Part B — Code prediction (30 pts)
3. Part C — Debugging challenges (30 pts)
4. Part D — Implementation exercises (30 pts)
5. Part E — Architecture and engineering decisions (20 pts)
6. Answer key
7. Scoring rubric
8. Promotion criteria and remediation

Part A — Conceptual questions (40 pts)

20 questions, 2 points each. Answer in 2–5 sentences. The lesson each question targets is in brackets (hidden from view only by your discipline: do not peek if you want a clean diagnostic).

A1 [01]. A page animates a box with `left` changes in JavaScript and, separately, with a CSS `transform` animation. A 400 ms long task runs on the main thread. Which animation is likely to keep moving, and

why?

A2 [02]. Why can the Elements panel show markup that differs from "View Source"? Name two things the browser does that cause the difference, and say what the render tree contains that the DOM does not exclude (or vice versa).

A3 [03]. For each change, state the earliest rendering stage that must re-run: (a) changing `background-color`, (b) changing `width`, (c) changing `transform` on a layer the compositor can animate.

A4 [04]. `document.getElementsByClassName("row")` and `document.querySelectorAll(".row")` both return collections. What is the behavioral difference that can turn a simple loop into an infinite loop, and why is setting `innerHTML` with user input dangerous?

A5 [05]. Explain what happens to `var`, `let`, and function declarations during the creation phase of an execution context, and what the *temporal dead zone* is.

A6 [06]. Define a closure precisely. Does a closure capture the *value* or the *variable*? Give a one-sentence consequence for event handlers created in a loop.

A7 [07]. What does `===` compare for objects and functions? How does `Object.is` differ from `===` for primitives? Why does it matter that a function expression creates a new function object each time it is evaluated?

A8 [08]. Why does `const next = { ...state }` not make updating `next.user.name` safe? What are two ways to produce a correct immutable update of a nested field?

A9 [09]. List the four ways the value of `this` is determined for a non-arrow function call, and explain why `const f = obj.method; f()` often loses `this`. Give two fixes.

A10 [10]. What properties of ES module syntax make tree shaking possible? Name two code patterns that defeat it.

A11 [11]. Describe the order in which an event loop iteration processes: one task, microtasks, and a rendering opportunity. Why does a long chain of resolved promises still block painting?

A12 [12]. What does an `async` function return? What happens if it throws? Explain the difference in timing and failure behavior between `await a(); await b();` and `await Promise.all([a(), b()])`.

A13 [13]. Distinguish `event.target` from `event.currentTarget`; `stopPropagation()` from `preventDefault()`. Name one situation where event delegation does **not** work out of the box and the alternative.

A14 [14]. Name three facts about `fetch` that commonly surprise beginners (error semantics, cancellation, cross-origin). Why can a CORS failure not be fixed purely in client code?

A15 [15]. Explain the HTTP caching strategy you would use for a SPA's `index.html` versus its content-hashed JS assets. What does `Cache-Control: no-cache` mean (as opposed to `no-store`)?

A16 [16]. Define "garbage" for a JavaScript engine. Give three distinct leak sources in a browser app and the reference chain that keeps the object alive in one of them.

A17 [17]. In a flame chart, what is the difference between self time and total time, and which one do you sort by to find the hot function? Why can debouncing not fix a slow event handler?

A18 [18]. What does it mean that TypeScript types are erased? Why is `unknown` preferable to `any` for parsed JSON, and why is `if (x)` an unsafe narrowing for `number | undefined`?

A19 [19]. Why is `{ status: "success"; data: T } | { status: "error"; error: Error } | { status: "loading" }` better than `{ loading: boolean; data?: T; error?: Error }`? Why is `children: React.ReactNode` better than `children: JSX.Element`?

A20 [20]. A build succeeds but production crashes with a type-related `TypeError`. Give two reasons the build did not catch it. Also: are `VITE_`-prefixed environment variables secret?

Part B — Code prediction (30 pts)

10 questions, 3 points each (2 points for the correct output, 1 point for the correct reasoning). Write the **exact output** in order. If a line throws, say what and where.

B1. Event loop ordering.

```
console.log("A");
setTimeout(() => console.log("B"), 0);
Promise.resolve().then(() => console.log("C"));
queueMicrotask(() => console.log("D"));
(async () => {
  console.log("E");
  await null;
  console.log("F");
})();
console.log("G");
```

B2. Closures in loops.

```
const fns = [];
for (var i = 0; i < 3; i++) fns.push(() => i);
const gns = [];
for (let j = 0; j < 3; j++) gns.push(() => j);
console.log(fns.map((f) => f()), gns.map((g) => g()));
```

B3. References and copies.

```
const a = { x: 1 };
const b = { ...a };
const c = a;
c.x = 2;
console.log(a.x, b.x, a === c, a === b);
```

B4. `this`.

```
const o = { n: 1, f() { return this?.n; } };
const f = o.f;
const g = o.f.bind({ n: 9 });
console.log(o.f(), f(), g());
```

(Assume this runs in a module, so strict mode applies. Would the output change in sloppy mode? Say why.)

B5. Hoisting and the temporal dead zone.

```
console.log(typeof x);
console.log(typeof y);
var x = 1;
let y = 2;
```

B6. Event propagation. The markup is `<div id="outer"><button id="btn">Go</button></div>`.

```
outer.addEventListener("click", () => console.log("outer-capture"), true);
btn.addEventListener("click", () => console.log("button"));
outer.addEventListener("click", () => console.log("outer-bubble"));
btn.click();
```

Then: what is printed if the button's handler also calls `event.stopPropagation()`?

B7. TypeScript narrowing (give the printed output and the return type of `f`).

```
function f(x: string | number | null) {
  if (!x) return "falsy";
  return typeof x === "string" ? x.length : x.toFixed(1);
}
console.log(f(""), f(0), f("ab"), f(2), f(null));
```

B8. Promise combinators (inside an `async` function).

```
const r = await Promise.allSettled([Promise.resolve(1), Promise.reject(new Error("x"))]);
console.log(r.map((s) => s.status), r[1].reason.message);
```

B9. Reachability. After `fn()` returns and the engine has run a full GC, which of `a`, `b`, `c` can be collected?

```
const cache = new Map();
const weak = new WeakMap();
function fn() {
  const a = {};
  const b = {};
  const c = {};
  cache.set("a", a);
  weak.set(b, "meta");
  return c.constructor === Object; // c is only used locally
}
fn();
```

B10. Module live bindings.

```
// counter.js
export let count = 0;
export function inc() { count++; }

// main.js
import { count, inc } from "./counter.js";
inc();
console.log(count);
count++;
```

Part C — Debugging challenges (30 pts)

6 challenges, 5 points each: **2 points** for identifying the root cause, **1 point** for stating how you would confirm it with a tool or experiment, **2 points** for a correct fix (including relevant edge cases).

C1. The ticking leak. (Usage-level React: `useEffect`'s returned function is its cleanup, run on unmount and before re-running. Details come in Phase 3.)

```
function Ticker() {
  const [now, setNow] = useState(Date.now());
  useEffect(() => {
    setInterval(() => setNow(Date.now()), 1000);
  }, []);
  return <time>{now}</time>;
}
```

After navigating between routes 50 times the tab's memory grows and the CPU is busy even on pages that do not show `Ticker`. In development with Strict Mode you notice two updates per second right after mount. Find the bug, explain the symptoms, state how you would confirm in DevTools, and fix it.

C2. The "user" that is not a user.

```
async function loadUser(id) {
  const res = await fetch(`/api/users/${id}`);
  const user = await res.json();
  return user;
}
```

In production, for a deleted user the app shows `undefined` fields in one environment and a console error `Unexpected token '<'` in another. Explain both symptoms and fix the function (including cancellation support).

C3. The update that did not update.

```
function addTodo(state, text) {
  state.todos.push({ id: Date.now(), text });
  return state;
}
const s1 = { todos: [] };
const s2 = addTodo(s1, "write tests");
if (s1 === s2) console.log("no change detected");
```

A UI layer that compares `prev === next` to decide whether to update never updates. Explain why and rewrite `addTodo` immutably. Mention one edge case about id generation.

C4. The slow resize.

```
for (const row of rows) {
  row.style.height = row.scrollHeight + "px";
}
```

With 3,000 rows this takes ~1.2 s on a laptop; the Performance panel shows hundreds of purple "Layout" blocks inside one task. Explain, and rewrite. State one condition under which your rewrite could change behavior.

C5. The compile error that is also a behavior bug.

```
interface User { id: number; nickname?: string }
declare const users: User[];

const names = users.map((u) => u.nickname);          // (string | undefined)[]
const lengths = names.filter(Boolean).map((n) => n.length);
//                                                    ^ error: 'n' is possibly 'undefined'
```

Explain the compile error, then explain a *runtime* behavior difference that `filter(Boolean)` has compared with `filter((n) => n !== undefined)` for this data, and write a correct, type-safe version.

C6. "Fixed" the env error.

```
// src/payments.ts (Vite + React app)
const key = process.env.STRIPE_SECRET_KEY;
```

In the browser console: `ReferenceError: process is not defined`. A teammate "fixes" it by renaming the variable to `VITE_STRIPE_SECRET_KEY` and changing the code to `import.meta.env.VITE_STRIPE_SECRET_KEY`. The error disappears. Explain what is wrong with this fix and describe the correct design.

Part D — Implementation exercises (30 pts)

5 exercises, 6 points each: **3** correctness on the main behavior, **1** edge cases, **1** types/API design, **1** cleanup/resource handling or complexity. Write real, runnable TypeScript.

D1. `debounce` **with** `cancel` **and** `flush`**.** Signature: `debounce<A extends unknown[]>(fn: (...args: A) => void, waitMs: number)`; the returned function has `.cancel()` (drop pending call) and `.flush()` (run the pending call immediately if any). Trailing-edge semantics, using the latest arguments.

D2. Immutable nested update. Given

```
interface User { id: string; name: string; tags: readonly string[] }
interface State {
  users: { byId: Readonly<Record<string, User>>; ids: readonly string[] };
  meta: { lastEditedId: string | null; revision: number };
}
```

write `renameUser(state: State, id: string, name: string): State` that returns the **same state object** if nothing changes (unknown id or same name), otherwise a new state where only the changed path has new references, `meta.lastEditedId` is set, and `meta.revision` is incremented. `tags` must keep its original reference.

D3. `Result` **type with exhaustive helpers.** Define `Result<T, E>` as a discriminated union (`ok` / `err`) and implement `map`, `unwrapOr`, and `match(result, { ok, err })` with correct generics and an exhaustiveness check.

D4. Event delegation helper. Implement

```
function delegate<K extends keyof HTMLElementEventMap>(
  root: HTMLElement,
  selector: string,
  type: K,
  handler: (event: HTMLElementEventMap[K], matched: Element) => void,
): () => void  // returns an unsubscribe function
```

It must find the nearest ancestor of `event.target` matching `selector` **within** `root`, ignore matches outside `root`, and handle the case where `event.target` is not an `Element`.

D5. `fetchJson` **with timeout and cancellation.**

```
function fetchJson<T>(
  url: string,
  options: { signal?: AbortSignal; timeoutMs?: number; validate: (data: unknown) => data is T
},
): Promise<T>
```

Rejects with distinguishable errors for: HTTP error status, validation failure, timeout, and caller abort. Must not leak timers or listeners.

Part E — Architecture and engineering decisions (20 pts)

4 questions, 5 points each: **1** options considered, **2** criteria and reasoning, **1** a clear decision, **1** measurement or risk plan. Aim for 150–300 words each.

E1. The 3 MB bundle. A SPA's main JS bundle is 3 MB (≈ 900 KB gzipped). Mid-range phones show a 6-second blank screen on 4G. The team uses Vite + React + TypeScript. Describe your investigation and a prioritized plan. Include which tools you use, what you expect to find, and how you would prove the improvement to users, not just in the lab.

E2. The eight-hour dashboard. A monitoring dashboard is left open for entire shifts. Users report that after 4–6 hours the tab slows down and sometimes crashes. Production telemetry cannot reproduce it locally in short sessions. Design the investigation and the guardrails you would add.

E3. Adopting TypeScript strictness. A 150k-line codebase has `strict: false` and widespread `any`. Leadership wants "type safety" in a quarter. Propose a migration strategy, what you will *not* promise, and how you will keep the types honest at runtime boundaries.

E4. Deploys that break open tabs. After each deployment, a fraction of users see white screens or "Failed to fetch dynamically imported module". Explain the mechanism and design the build, caching, and client-side behavior that eliminate it.

---

6. Answer key

> Attempt every part before reading this section.

Part A — key (2 pts each: 1 point for the core fact, 1 point for the mechanism or consequence)

A1. The CSS `transform` animation (if it is compositor-eligible, which `transform` and `opacity` generally are) is likely to keep animating: the compositor thread can update layer transforms without the main

thread. The `left`-animation via JavaScript needs main-thread JS to set values *and* triggers style/layout/paint on the main thread, so it stalls for the 400 ms. Caveat to mention for full credit: compositor animation depends on browser behavior and the element being promoted to its own layer. *Common wrong answer:* "neither moves, JS is single-threaded"; that confuses the JS thread with the browser's other threads (Lesson 01).

A2. The DOM is the parser's *result*, not the source text. Causes: error recovery (e.g., misnested tags are repaired; `<tbody>` inserted into tables; unclosed `<p>` closed) and script/DOM mutations after load (JS-created nodes, framework rendering). View Source shows the original response bytes. The render tree contains only what will be rendered: the DOM includes `display: none` elements and non-visual nodes like `<head>`, which the render tree omits, and the render tree includes generated content (such as `::before` boxes) that has no DOM node (Lesson 02). Either direction of the DOM/render-tree difference earns the second point if explained.

A3. (a) `background-color`: style recalculation, then paint (and composite); no layout. (b) `width`: style → layout → paint → composite (layout affects geometry of other boxes). (c) `transform` on a promoted layer: composite only (style recalculation still determines the new value, but layout and paint are skipped). 2 pts: all three correct; 1 pt: two correct (Lesson 03).

A4. `getElementsByClassName` returns a **live** `HTMLCollection` that updates as the DOM changes; `querySelectorAll` returns a **static** `NodeList` snapshot. A loop like `for (const el of liveCollection) container.append(el.cloneNode())` where the clone also matches adds elements to the collection as it iterates, which never terminates. `innerHTML` with user input parses markup and can execute attacker-controlled script via event-handler attributes etc. (XSS); use `textContent` or sanitize (Lesson 04).

A5. Creation phase: `var` declarations are hoisted and initialized to `undefined`; function declarations are hoisted *with* their bodies (callable before the declaration); `let` / `const` / `class` bindings are created but **uninitialized**: accessing them before their declaration executes throws a `ReferenceError`. That interval is the temporal dead zone. Note `typeof` on a TDZ binding also throws (Lesson 05).

A6. A closure is a function together with a reference to the lexical environment in which it was created; the function can access the *variables* of enclosing scopes even after those scopes finished. It captures **variables (bindings)**, not copies of values. Consequence in loops: with `var` all handlers share one variable and see its final value; `let` creates a fresh binding per iteration so each handler sees its own (Lesson 06).

A7. For objects and functions `===` compares **identity** (same reference), not contents: `{} === {}` is `false`. `Object.is` differs from `===` for `NaN` (`Object.is(NaN, NaN)` is `true`, `NaN === NaN` is `false`) and signed zeros (`Object.is(0, -0)` is `false`, `0 === -0` is `true`). Each evaluation of a function expression creates a **new** function object, so a function recreated per call is never identical to the previous one; this is what makes "did the dependency change?" checks (and later `React.memo`/dependency arrays) fire unexpectedly (Lesson 07).

A8. Spread is **shallow**: `next.user` is the same object as `state.user`, so assigning `next.user.name` mutates the original. Correct: `{ ...state, user: { ...state.user, name } }` (copy along the path), or use a library that produces structural sharing (Immer); `structuredClone` makes a full deep copy (correct but wasteful, and it cannot clone functions) (Lesson 08).

A9. (1) Called as a method `obj.m()`: `this` is `obj`. (2) Plain call `f()`: `undefined` in strict mode (modules/classes), `globalThis` in sloppy mode. (3) `new F()`: `this` is the new object. (4) Explicit: `call` / `apply` / `bind`. (Arrow functions have no own `this`; they use the enclosing lexical `this`.) `const f = obj.method; f()` loses the receiver because the call-site no longer has `obj.` before it. Fixes: `obj.method.bind(obj)`, an arrow wrapper `() => obj.method()`, or define the method as a class field arrow function (with the memory trade-off of one function per instance) (Lesson 09).

A10. ES module `import` / `export` are **static** (declared at top level, specifiers are string literals), so a bundler can build the full graph and know which exports are used without executing code; bindings are live and modules are evaluated once. Patterns that defeat tree shaking: CommonJS (`require` / `module.exports` are dynamic), modules with top-level side effects, `import * as ns` followed by dynamic property access (`ns[name]`), barrel files re-exporting side-effectful modules, and packages missing/incorrect `"sideEffects"` metadata (Lesson 10, Lesson 20).

A11. One iteration: pick a task and run it to completion; then drain the **entire** microtask queue (including microtasks queued by microtasks); then, if it is a rendering opportunity (not every iteration), run "update the rendering" steps (rAF callbacks, style, layout, paint). A chain of already-resolved promises keeps adding microtasks, and the queue must be empty before the loop proceeds to rendering or the next task, so rendering and input starve (Lesson 11).

A12. An `async` function always returns a **promise**; `return v` fulfills it with `v` and a `throw` rejects it (synchronous throws inside become rejections, never synchronous exceptions to the caller). `await a(); await b();` runs sequentially (b starts after a completes) and a failure in `a` prevents `b` from starting. `Promise.all([a(), b()])` starts both immediately and completes when both fulfill, rejecting as soon as the first rejects (the other keeps running; its result is ignored). `allSettled` waits for all and never rejects (Lesson 12).

A13. `target` is the element that originated the event (deepest), `currentTarget` is the element whose listener is running. `stopPropagation()` prevents further propagation to other nodes' listeners; `preventDefault()` cancels the browser's default action (navigation, form submit) without stopping propagation. Delegation fails for events that do not bubble (`focus` / `blur`, `mouseenter` / `mouseleave`, `scroll` on elements, media events): use capture-phase listeners or the bubbling counterparts `focusin` / `focusout`, or `mouseover` / `mouseout` with checks (Lesson 13).

A14. (1) `fetch` rejects only on network-level failure; HTTP 404/500 *resolve* with `res.ok === false`, so you must check `res.ok`. (2) Cancellation uses `AbortController` / `AbortSignal`; aborting rejects with an `AbortError`. (3) Cross-origin requests are governed by CORS: the *server* must send the right headers, and non-simple requests trigger a preflight; the browser enforces it. A CORS failure cannot be fixed client-side because the policy decision is made by the browser based on server headers; `mode: "no-cors"` only yields opaque responses. Fix on the server, or proxy through your own origin (Lesson 14).

A15. `index.html`: must be revalidated so new deploys are picked up: `Cache-Control: no-cache` (store, but revalidate with ETag/Last-Modified each time → cheap 304s). Hashed assets (`app.3fa91c.js`): `public, max-age=31536000, immutable` because the URL changes whenever content changes. `no-cache` does **not** mean "do not cache": it means "may store, but must revalidate before reuse". `no-store` means do not store at all (sensitive data) (Lesson 15).

A16. Garbage = memory occupied by objects **unreachable** from roots (globals, stack, active host references like timers/listeners, DOM). Leak sources: (1) timers never cleared, (2) event listeners/observers on long-lived targets never removed, (3) module-level caches/arrays growing without bound, (4) detached DOM nodes held by variables, (5) subscriptions/sockets not closed, (6) closures retaining big data. Example chain: `window → resize listener → closure context → detached panel element + rows array`. 1 pt for definition, 1 pt for source and chain (Lesson 16).

A17. Self time: time in the function's own code; total time includes callees. Sort **bottom-up by self time** to find hot spots (use the call tree/total time to find which entry point is expensive). Debouncing reduces *how often* a handler runs, not *how long* each run blocks the thread; a debounced 600 ms handler still freezes the UI for 600 ms when it runs (Lesson 17).

A18. Type annotations and interfaces are removed during compilation; nothing validates data at runtime, so claims about external data (`JSON.parse`, `fetch`) are unchecked. `unknown` forces narrowing before use (`any` silently disables checking and spreads). `if (x)` removes *all* falsy values including a legitimate `0` (and `""` / `NaN`), so a counter of `0` is treated as absent; use `x !== undefined` (Lesson 18).

A19. The union makes impossible states unrepresentable (no "loading with error"), gives each state only the data valid for it, supports exhaustive switches that fail compilation when a state is added, and narrows access (`data` exists only in `success`). The boolean-plus-optionals bag admits 8 combinations of which ~3 are meaningful. `ReactNode` covers everything renderable (strings, numbers, `null`, arrays, elements); `JSX.Element` rejects text, numbers, `null`, and arrays (Lesson 19).

A20. Reasons the build passes: (1) transpilers (esbuild/SWC/Babel) strip types **without** checking them and `tsc --noEmit` is not in the pipeline; (2) types are erased and external data/`any` / `as` / `!` can violate them at runtime (also type declarations that lie). `VITE_` variables are **not** secret: they are inlined into public client JavaScript at build time (Lesson 20).

Part B — key

B1. Output, one per line: `A`, `E`, `G`, `C`, `D`, `F`, `B`. Reasoning: synchronous code first: `A`, then the async IIFE runs synchronously up to its first `await` (prints `E`), then `G`. Microtasks are queued in order: the `.then` callback (C), `queueMicrotask` (D), and the continuation after `await null` (F), because `await null` queues its continuation when the IIFE executes (after C and D were queued). They run before the timer task, so `C D F`, then the timer callback `B`. Scoring: 2 pts for the exact sequence (1 pt if only `F` / `C` / `D` order is wrong), 1 pt for the reason (sync, microtasks drain, then tasks) (Lesson 11, 12).

B2. `[ 3, 3, 3 ] [ 0, 1, 2 ]`. `var i` is function/global-scoped: one binding shared by all closures, which see its final value 3. `let j` creates a new binding per iteration (Lesson 06).

B3. `2 1 true false`. `c` aliases `a`, so `c.x = 2` changes `a.x`. `b` was copied before the mutation (shallow copy), so `b.x` stays `1`. `a === c` same reference; `a === b` different objects (Lesson 08).

B4. `1 undefined 9`. `o.f()` has receiver `o` (`1`). `f()` is a plain call: in strict mode `this` is `undefined`, so `this?.n` is `undefined`. The bound function uses `{ n: 9 }` (`9`). In sloppy mode a plain call's `this` is `globalThis`, and `globalThis.n` is `undefined` (unless a global `n` exists), so the output is the same unless a global `n` is defined, an example of why sloppy mode hides bugs (Lesson 09).

B5. Prints `undefined`, then throws `ReferenceError: Cannot access 'y' before initialization`. `var x` is hoisted and initialized to `undefined`, so `typeof x` is `"undefined"` (the string is printed). `let y` is in the temporal dead zone; even `typeof` throws. Common wrong answer: "prints `undefined` twice" (`typeof` on an *undeclared* identifier is safe, but a TDZ binding is declared) (Lesson 05).

B6. `outer-capture`, `button`, `outer-bubble`. Dispatch goes capture (root → target), target, bubble (target → root); capture listeners on ancestors run first, and `btn.click()` dispatches a real, bubbling click event. With `stopPropagation()` in the button's handler: `outer-capture`, `button` (the bubble listener on `outer` does not run; the capture listener already ran before the target phase) (Lesson 13).

B7. Output: `falsy falsy 2 2.0 falsy`. The return type of `f` is `string | number` (`"falsy"` is `string`, `x.length` is `number`, `x.toFixed(1)` is `string`). `""` and `0` are falsy, so they take the first return. After `!x` returns, `x` is `string | number` (truthy); `"ab"` → length `2`; `2` → `"2.0"`. Common error: expecting `f("")` to reach the `typeof` branch (Lesson 18).

B8. `[ 'fulfilled', 'rejected' ] x`. `allSettled` never rejects; each entry is `{ status: "fulfilled", value }` or `{ status: "rejected", reason }` (Lesson 12).

B9. `b` and `c` are collectable; `a` is not. `a` is strongly referenced by the module-level `Map`; `b` is only a `WeakMap` key (weak), and `c` was a local. (The line `c.constructor === Object` is irrelevant noise: it does not retain `c`.) (Lesson 16).

B10. Prints `1`, then `count++` throws `TypeError: Assignment to constant variable` (the exact message varies by engine). Imports are **live, read-only views** of the exporting module's binding: `inc()` mutated the exporter's variable and `count` in `main.js` reflects it (1); importers cannot assign to imported bindings. Scoring: 1 pt for `1`, 1 pt for the throw, 1 pt for the live-binding reasoning (Lesson 10).

Part C — key

C1. *Root cause:* the interval ID from `setInterval` is never stored or cleared; the effect returns no cleanup. The timer (a host root) holds the callback, which closes over `setNow` and thereby the component's state machinery, so unmounted instances keep ticking and retaining memory; each mount adds another interval (Strict Mode's mount → cleanup → mount in development makes it visible as two intervals immediately). *Confirm:* Performance panel with the memory checkbox (CPU activity every second on unrelated routes, rising heap/listener-like counts), or `console.log` in the callback after unmounting; heap snapshot comparison shows repeated retained closures/timers; in dev, two ticks per second. *Fix:*

```
useEffect(() => {
  const id = window.setInterval(() => setNow(Date.now()), 1000);
  return () => window.clearInterval(id);
}, []);
```

Edge cases for full credit: using the functional or latest-value pattern if the callback depends on props; `useState(Date.now())` evaluates `Date.now()` every render (use `useState(() => Date.now())` or `useState(Date.now)` for lazy init: a minor but real point). A quick verification: unmount the component and confirm no ticks continue in the console or Performance recording (Lesson 16, Phase 3/06).

C2. *Root cause:* `fetch` does not reject on HTTP errors and the code never checks `res.ok`, and it assumes the body is JSON. Environment A: the API returns a 404 with a JSON error body such as `{ "error": "not found" }`; `res.json()` succeeds and the object is returned as if it were a user, so UI fields read `undefined`. Environment B: a proxy/CDN returns an HTML 404 page; `res.json()` throws `SyntaxError: Unexpected token '<'`. *Fix:*

```
async function loadUser(id: string, signal?: AbortSignal): Promise<User> {
  const res = await fetch(`/api/users/${encodeURIComponent(id)}`, { signal });
  if (!res.ok) throw new Error(`Failed to load user ${id}: HTTP ${res.status}`);
  const data: unknown = await res.json();
  if (!isUser(data)) throw new Error("Unexpected user payload");
  return data;
}
```

Full credit needs: `res.ok` check, `signal` passed through, optionally content-type check or validation, caller handles `AbortError` (Lesson 14, 18).

C3. *Root cause:* `push` mutates the existing array and `return state` returns the same object, so `s1 === s2` is `true`; reference-comparison change detection cannot see it. *Fix:*

```
function addTodo(state, text) {
  return { ...state, todos: [...state.todos, { id: crypto.randomUUID(), text }] };
}
```

Edge case: `Date.now()` ids collide when two todos are created in the same millisecond; use `crypto.randomUUID()` (secure contexts) or a counter. Also the function should be pure: generate ids outside or inject them for testability (Lesson 08).

C4. *Root cause:* **layout thrashing** (forced synchronous layout): each iteration reads `scrollHeight` (which needs up-to-date layout) after the previous iteration's write invalidated layout, forcing a full layout per row (~3,000 layouts). *Fix:* batch reads, then writes:

```
const heights = rows.map((row) => row.scrollHeight);
rows.forEach((row, i) => { row.style.height = heights[i] + "px"; });
```

Behavior change condition: if setting one row's height affects other rows' `scrollHeight` (e.g., rows share a flex/grid container whose layout depends on sibling sizes, or content reflows), the batched reads use pre-change values; then use two-phase logic (write what is independent first, then read) or CSS (`height: fit-content`/grid) instead of JS. Confirmation: the Performance panel shows repeated Layout events within a single task (Lessons 03, 04, 17).

C5. *Compile error:* `filter(Boolean)` is typed to return the same element type (`(string | undefined)[]`): the compiler does not understand that `Boolean` removes `undefined` (inferred type predicates in TS 5.5+, apply to arrow functions such as `n => n !== undefined`, not to `Boolean`). *Runtime difference:* `filter(Boolean)` removes **all falsy values**, including the empty string `""`; a user with `nickname: ""` is dropped, whereas `n !== undefined` keeps `""`, so `lengths` would differ (`0` is missing from the first result). *Fix:*

```
const lengths = names
  .filter((n): n is string => n !== undefined)
  .map((n) => n.length);
```

(Whether empty nicknames should count is a product decision; the point is to make the choice explicit.) (Lesson 18.)

C6. *Root cause of original error:* the browser has no `process`; Node-style `process.env` is not available client-side. *Why the fix is wrong:* `VITE_`-prefixed variables are statically inlined into the public JavaScript bundle; a **secret** key is now shipped to every visitor (and appears in source maps/caches). Renaming silenced the symptom and created a security incident (the key must be rotated). *Correct design:* secret keys exist only on the server; the client uses the **publishable** key (`VITE_STRIPE_PUBLISHABLE_KEY`) and calls your backend endpoint to create payment intents/sessions; add CI or code-review checks that reject `VITE_*SECRET*`-style names. Mention rotating the leaked key and checking logs (Lesson 20).

Part D — key (reference solutions; accept alternatives meeting the criteria)

D1.

```ts
interface Debounced<A extends unknown[]> {
  (...args: A): void;
  cancel(): void;
  flush(): void;
}

export function debounce<A extends unknown[]>(
  fn: (...args: A) => void,
  waitMs: number,
): Debounced<A> {
  let timer: ReturnType<typeof setTimeout> | undefined;
  let pending: A | undefined;

  const run = () => {
    timer = undefined;
    const args = pending;
    pending = undefined;
    if (args) fn(...args);
  };
  const debounced = ((...args: A) => {
    pending = args;
    if (timer !== undefined) clearTimeout(timer);
    timer = setTimeout(run, waitMs);
  }) as Debounced<A>;
  debounced.cancel = () => {
    if (timer !== undefined) clearTimeout(timer);
    timer = undefined;
    pending = undefined;
  };
  debounced.flush = () => {
    if (timer !== undefined) { clearTimeout(timer); run(); }
  };
  return debounced;
}
```

Criteria: latest args used; `cancel` clears the timer *and* the pending args; `flush` is a no-op when nothing is pending; no timer leak; generic argument tuple typing. Accept a `this`-preserving version if explained. The single `as Debounced<A>` is a justified contained cast (function with attached properties).

D2.

```ts
export function renameUser(state: State, id: string, name: string): State {
  const user = state.users.byId[id];
  if (user === undefined || user.name === name) return state;
  return {
    ...state,
    users: {
      ...state.users,
      byId: { ...state.users.byId, [id]: { ...user, name } },   // tags keeps its reference
via spread
    },
    meta: { ...state.meta, lastEditedId: id, revision: state.meta.revision + 1 },
  };
}
```

Criteria: returns the same reference for no-ops; `users.ids` retains its reference (not copied); `tags` reference preserved (shallow spread of `user`); `meta` copied with updates; no mutation (verify with `Object.freeze` in tests). Edge: `state.users.byId[id]` with `noUncheckedIndexedAccess` is `User | undefined` (good), and without it TypeScript types it as `User`, which the `=== undefined` comparison still guards at runtime (note the type lie). Prototype-key hazard: an id like `"__proto__"` / `"constructor"` on a plain object; use `Object.hasOwn` /a `Map` if ids are untrusted.

D3.

```ts
export type Result<T, E> = { ok: true; value: T } | { ok: false; error: E };

export const ok = <T>(value: T): Result<T, never> => ({ ok: true, value });
export const err = <E>(error: E): Result<never, E> => ({ ok: false, error });

export function map<T, U, E>(r: Result<T, E>, f: (t: T) => U): Result<U, E> {
  return r.ok ? ok(f(r.value)) : r;
}
export function unwrapOr<T, E>(r: Result<T, E>, fallback: T): T {
  return r.ok ? r.value : fallback;
}
export function match<T, E, R>(
  r: Result<T, E>,
  handlers: { ok: (value: T) => R; err: (error: E) => R },
): R {
  switch (r.ok) {
    case true:  return handlers.ok(r.value);
    case false: return handlers.err(r.error);
    default: { const _x: never = r; return _x; }
  }
}
```

Criteria: the discriminant is a literal (`true` / `false`); `map` returns the original error variant unchanged (type-correct because `Result<never, E>`-shaped values are assignable); `match` has an exhaustiveness `never` branch. In `map`, `return r` in the false branch has type `{ ok: false; error: E }` which is assignable to `Result<U, E>`. Accept `if/else` instead of `switch`.

D4.

```
export function delegate<K extends keyof HTMLElementEventMap>(
  root: HTMLElement,
  selector: string,
  type: K,
  handler: (event: HTMLElementEventMap[K], matched: Element) => void,
): () => void {
  const listener = (event: Event) => {
    const target = event.target;
    if (!(target instanceof Element)) return;                // e.g., text nodes in old
browsers, document, window
    const matched = target.closest(selector);
    if (matched && root.contains(matched)) {
      handler(event as HTMLElementEventMap[K], matched);
    }
  };
  root.addEventListener(type, listener);
  return () => root.removeEventListener(type, listener);
}
```

Criteria: uses `closest` (handles clicks on inner elements); checks `root.contains(matched)` (the selector could match an ancestor *outside* `root`); guards non-`Element` targets; returns an unsubscribe that removes the same function reference; the one cast is contained and explained (the listener receives `Event`). Edge: `closest` also matches `target` itself; `root` itself may match the selector (decide whether to exclude `matched === root`); non-bubbling events (`focus`) need `focusin` or capture (Lesson 13).

D5.

```ts
export class HttpError extends Error { constructor(readonly status: number) { super(`HTTP ${status}`); } }
export class ValidationError extends Error {}
export class TimeoutError extends Error {}

export async function fetchJson<T>(
  url: string,
  { signal, timeoutMs = 10_000, validate }: {
    signal?: AbortSignal; timeoutMs?: number; validate: (d: unknown) => d is T;
  },
): Promise<T> {
  const controller = new AbortController();
  let timedOut = false;
  const timer = setTimeout(() => { timedOut = true; controller.abort(); }, timeoutMs);
  const onAbort = () => controller.abort();
  if (signal?.aborted) controller.abort(); else signal?.addEventListener("abort", onAbort, { once: true });

  try {
    const res = await fetch(url, { signal: controller.signal });
    if (!res.ok) throw new HttpError(res.status);
    const data: unknown = await res.json();
    if (!validate(data)) throw new ValidationError(`Invalid payload from ${url}`);
    return data;
  } catch (e) {
    if (timedOut) throw new TimeoutError(`Timed out after ${timeoutMs} ms`);
    throw e;                                          // caller abort surfaces as AbortError (DOMException)
  } finally {
    clearTimeout(timer);
    signal?.removeEventListener("abort", onAbort);
  }
}
```

Criteria: distinguishes the four failure kinds; combines caller signal and timeout via one controller; handles an already-aborted signal; **always** clears the timer and removes the listener in `finally`; `res.ok` checked; validation before returning `T`. Accept `AbortSignal.any` / `AbortSignal.timeout` if the answer notes browser-support requirements (⚠ verify support on MDN). Edge: if the timeout fires *after* the response body began but before `json()` finishes, abort rejects `res.json()`, mapped to `TimeoutError` by the flag.

Part E — key (evaluation criteria and exemplar reasoning)

E1. Exemplar. *Investigate first:* run a production build with a bundle analyzer (Phase 7/16) to see what the 3 MB consists of; check source-map-explorer-style attribution; look for duplicate libraries, whole-library imports (moment locales, lodash non-ESM), chart/editor libraries on the entry path, unminified or dev-

mode code, and polyfills for old targets. Measure with Lighthouse and WebPageTest-style throttling plus real-user data (LCP, TTI-related, INP) segmented by device class. Check the network waterfall: a blank screen for 6 s on 4G often combines download (900 KB gz), parse/compile on a mid-range CPU, and a waterfall of dependent requests. *Plan (priority by expected impact/effort):* (1) remove/replace oversized dependencies and fix imports so tree shaking works; (2) route-level code splitting with `React.lazy` and lazy-load heavy widgets; (3) ensure production build, modern build target, compression (Brotli); (4) defer non-critical third-party scripts; (5) preload critical chunks, cache hashed assets immutably; (6) consider SSR/streaming only if first paint remains the bottleneck after the above (cost: infrastructure). *Prove it:* a performance budget in CI (bundle size, lab LCP) plus RUM p75 LCP/INP for the mobile segment before/after rollout; expect lab gains to exceed field gains. *Scoring notes:* full marks need analysis before action, a prioritized list, awareness that parse/execute cost (not just bytes) matters on phones, and field validation. Wrong: "rewrite in framework X" without evidence (Lessons 17, 20).

E2. Exemplar. *Hypotheses:* leaked listeners/timers/subscriptions per widget or per navigation; unbounded in-memory history (ticks, logs, undo stacks); detached DOM; caches without eviction; a third-party chart library instances never destroyed; unbounded query/result caches. *Instrumentation:* add (sampled) counters: DOM node count, active timer/subscription/listener counts per feature, cache sizes, array lengths of history stores, and, where available, JS heap via `performance.memory` (Chromium only) or `measureUserAgentSpecificMemory()` (needs cross-origin isolation); send them with session age to your telemetry. Correlate heap growth with user actions (widget adds/removes, route changes). *Reproduce:* automated soak test (Playwright) that repeats suspect flows 200× with forced GC (CDP `HeapProfiler.collectGarbage`) and records post-GC heap; bisect flows until the slope appears. *Diagnose:* heap snapshot comparison (objects scaling with N), Detached filter, retainers → source line. *Guardrails:* disposer/`AbortController` patterns in shared utilities, bounded data structures (ring buffers/LRU), lint/review rules for effects without cleanup, soak test in CI with a slope threshold, alerts on memory counters p95, and a last-resort "refresh suggestion" UX for kiosks (not a substitute). *Scoring:* credit measurement over guessing, recognition that the post-GC floor is the signal, a reproduction strategy, and prevention (Lesson 16).

E3. Exemplar. *Strategy:* incremental ratchet. Add `tsc --noEmit` to CI immediately (even if informational), turn on `allowJs` / `checkJs` selectively, and make **new/changed files strict** via per-directory tsconfig `extends` or a strict-file allowlist; convert leaf modules first, boundaries (API clients, storage, message handlers) early because that's where wrong types do the most damage; enable flags in order of payoff: `noImplicitAny` → `strictNullChecks` → remaining `strict` → `noUncheckedIndexedAccess`; track metrics (count of `any`, `as`, `@ts-ignore`, files under strict) with a CI no-increase rule; forbid new `@ts-ignore` without `@ts-expect-error` + reason. *Runtime honesty:* validate external data at the edge (schema validators or guards) so internal types are trustworthy; avoid generating false confidence with casts. *Will not promise:* "type safety" meaning runtime safety, or completion of 150k lines of strict typing in a quarter; promise measurable ratchet milestones (e.g., all new code strict, API layer validated, `strictNullChecks` on for the top N modules, `any` reduced by X%). Acceptable alternatives: big-bang rewrite (rejected for risk) (Lessons 18, 19).

E4. Exemplar. *Mechanism:* a tab loaded the old `index.html`/entry chunk; the deploy replaced the hashed assets and removed old files (or CDN purge); when the old app lazy-loads a chunk by its old hashed URL, the server returns 404 (or the SPA fallback `index.html`, with a wrong MIME type) and the dynamic `import()` rejects. *Design:* (1) upload new hashed assets *before* switching `index.html`; (2) keep previous releases' assets for a retention window (e.g., 30 days) so old tabs can still load their chunks; (3)

`Cache-Control: no-cache` on `index.html`, `immutable` long max-age on hashed assets; (4) real 404s for missing `/assets/*` (do not rewrite to `index.html`); (5) client: wrap lazy loads with a one-time reload on failure plus an error boundary with a reload action; (6) optional: version endpoint polling to prompt a refresh; (7) rollbacks restore the previous `index.html` because old assets still exist. *Risk plan:* monitor `ChunkLoadError`/dynamic import failures per release; watch for reload loops (guard with a session flag); service workers need an update strategy or they will serve stale HTML (Lessons 15, 20).

---

7. Scoring rubric

7.1 Points and categories

Scores are tallied into **four categories**. A passing result requires **≥ 80% overall (120/150)** *and* **≥ 70% in every category**.

Category	What it measures	Parts	Max points	70% floor
Explanation	Can you state and justify the concepts precisely?	Part A	40	28
Reasoning	Can you predict behavior and make justified decisions?	Part B (30) + Part E (20)	50	35
Debugging	Can you find root causes and verify fixes?	Part C	30	21
Implementation	Can you write correct, typed, resource-safe code?	Part D	30	21
Total			**150**	**120 overall**

7.2 Per-item scoring rules

Part	Points per item	Breakdown
A	2	1 = core fact correct. 1 = mechanism/consequence/example correct. If the answer has the right words but a wrong mechanism, score at most 1.
B	3	2 = exact output (partial: 1 if one line/item order is wrong, 0 if the approach is wrong). 1 = correct reasoning (naming the actual rule: microtask draining, shared binding, TDZ, live binding, ...).
C	5	2 = root cause identified (symptom explanations count). 1 = a concrete confirmation method (tool/experiment). 2 = correct fix incl. relevant edge case (1 for a fix that works but ignores a named edge case).
D	6	3 = main behavior correct. 1 = edge cases handled (named in the key criteria). 1 = types/API design sound (no unnecessary `any`/casts). 1 = cleanup/resource/complexity handled (timers, listeners, signals, no accidental $O(n^2)$, no mutation). Code that does not run/compile can earn at most 3.

Part	Points per item	Breakdown
E	5	1 = alternatives considered. 2 = criteria and causal reasoning (not assertion). 1 = clear decision. 1 = measurement/risk plan.

Self-scoring discipline. Give yourself credit only for what you wrote *before* checking the key. Where the key lists multiple acceptable answers, any of them counts. When unsure, grade the lower score.

7.3 Result bands

Overall	Category floors met?	Result
≥ 135 (90%)	Yes	**Distinction:** proceed; you will likely find Phases 1–2 straightforward on the foundation side.
120–134	Yes	**Pass:** proceed to Phase 1. Revisit any lesson tied to your lowest item scores while reading Phase 1.
≥ 120	**No** (any category < 70%)	**Conditional:** you do not pass yet. Remediate the failing category (§8) and retake **only** that category's parts (with fresh variants, see below).
100–119	—	**Not yet:** targeted remediation (§8), then full retake after at least 7 days.
< 100	—	**Return to lessons:** redo exercises (not just reading) for the lessons mapped to your missed items, then full retake.

8. Promotion criteria and remediation

8.1 Promotion to Phase 1

You are ready for Phase 1 when **all** of the following hold:

1. **Score gate:** ≥ 120/150 overall and ≥ 70% in each category (§7).
2. **Critical-concept gate:** you scored at least 80% of the available points within each of the three "critical" clusters that Phases 1–3 rely on most:
   - *Closures and scope* (A5, A6, B2, B5),
   - *Identity and immutability* (A7, A8, B3, C3, D2),
   - *Event loop and async ordering* (A11, A12, B1, B8).
3. **Practice gate:** you can, without references, (a) explain why a heavy `onClick` freezes unrelated UI, (b) write an effect-style setup/cleanup pair that leaks nothing, and (c) model a loading/success/error state as a discriminated union.
4. **Toolchain gate:** you have run a Vite + React + TypeScript project locally, executed `vite build` and `vite preview`, and found your component code in the browser's Sources panel via source maps.

If you pass the score gate but miss a critical cluster, proceed to Phase 1 *only after* a 30-minute focused review of the lessons below, because those concepts are used from Phase 1/13 onward.

8.2 Remediation map

If you lost points on...	Re-study	Then do
A1, A2, A3, A4, C4	01, 02, 03, 04	Record a Performance trace of a layout-thrashing loop and its fix; compare Layout event counts
A5, A6, B2, B5	05, 06	Hand-trace three nested-closure examples on paper, then run them
A7, A8, A9, B3, B4, C3, D2	07, 08, 09	Write five immutable update functions for nested state; test with `Object.freeze`
A10, B10	10	Build a tiny two-module example demonstrating live bindings and a failed tree shake
A11, A12, B1, B8, D5	11, 12	Predict-then-run 10 ordering puzzles mixing timers, promises, and `queueMicrotask`
A13, B6, D4	13	Log all three phases on nested elements; implement delegation with `focusin`
A14, A15, C2, D5	14, 15	Inspect cache headers in the Network panel for a hashed vs unhashed asset
A16, B9, C1, E2	16	Reproduce the §4 listener leak and find it with snapshot comparison
A17, C4, D1, E1	17	Profile the filter-as-you-type scenario with 4× CPU throttling
A18, A19, B7, C5, D3	18, 19	Re-type a loose API client with unions and boundary validation
A20, C6, E3, E4	20	Deploy a Vite build twice and reproduce a chunk 404 with an old tab

8.3 Retake policy

- **Category retake:** if only one category failed its floor, retake the relevant parts after at least **3 days**. Use fresh variants: re-derive new problems from the remediation map exercises (change data, change shapes, change the failure mode) rather than re-reading the same items.
- **Full retake:** after at least **7 days** if overall < 120.
- **Do not** memorize the key; the goal is the ability to predict and diagnose in unfamiliar code. If a retake succeeds only because items were remembered, treat it as a failed attempt.

8.4 What this assessment does not cover

Phase 0 intentionally excludes React APIs. You are not expected to know `useState` semantics, reconciliation, or rendering phases yet. Where React appears (C1, §5 of lesson 19) it is used at *usage level* so you can recognize setup/cleanup and typing patterns. Those concepts are assessed in Phases 1–3.

Next: Phase 1 — React's purpose and philosophy, then Phase 1 assessment after Phase 1.